scientific reports

OPEN

Improving deep learning-based neural distinguisher with multiple ciphertext pairs for speck and Simon

Yufei Hou[1], Jie Liu[1]✉, Shouxu Han[2], Zhongjun Ma[3], Xi Ye[4] & Xuan Nie[1]

The neural network-based differential distinguisher has attracted significant interest from researchers due to its high efficiency in cryptanalysis since its introduction by Gohr in 2019. However, the accuracy of existing neural distinguishers remains limited for high-round-reduced cryptosystems. In this work, we explore the design principles of neural networks and propose a novel neural distinguisher based on a multi-scale convolutional block and dense residual connections. Two different ablation schemes are designed to verify the efficiency of the proposed neural distinguisher. Additionally, the concept of a linear attack is introduced to optimize the input dataset for the neural distinguisher. By combining ciphertext pairs, the differences between ciphertext pairs, the keys, and the differences between the keys, a novel dataset model is designed. The results show that the accuracy of the proposed neural distinguisher, utilizing the novel neural network and dataset, is 0.15–0.45% higher than Gohr's distinguisher for Speck 32/64 when using a single ciphertext pair as input. When using multiple ciphertext pairs as input, it is 1.24–3.5% higher than the best distinguishers for Speck 32/64 and 0.32–1.83% higher than the best distinguishers for Simon 32/64. Finally, a key recovery attack based on the proposed neural distinguisher using a single ciphertext pair is implemented, achieving a success rate of 61.8%, which is 9.7% higher than the distinguisher proposed by Gohr. Therefore, the proposed neural distinguisher demonstrates significant advantages in both accuracy and key recovery rate.

Keywords Neural distinguisher, Differential analysis, Key recovery attack, Deep learning

Deep learning is applied to various fields and has significantly advanced research in these areas, such as network security[1,2], data encryption[3], intelligent agriculture[4,5], and circuit design[6–9], among others. In 2019, Gohr[10] introduced the deep residual neural network into the cryptanalysis of Speck 32/64[11], achieving significant advantages over traditional cryptanalysis methods. In this work, Gohr verified the differential distribution model using the Markov model and trained a deep residual network on cipher pairs with specified input differences as well as random cipher pairs. Furthermore, a multi-stage key recovery attack on round-reduced Speck 32/64 was proposed to validate the efficiency of the proposed neural distinguisher (ND).

Gohr's work provides new ideas and methods for the cross-disciplinary study of cryptography and deep learning. Benamira et al.[12] compared the performance of deep learning-based distinguishers with traditional ones. They combined the multi-layer perceptron with a conventional neural network, which achieved higher accuracy. They pointed out that neural networks not only rely on the distribution of ciphertext but also use the internal state difference of the penultimate or third-to-last round. Bellini et al.[13] compared traditional and deep learning-based distinguishers, finding that using a single difference in the input simplifies training and improves accuracy. Tian et al.[14] theoretically proved that the SIMON cryptosystem is not a Markov cryptosystem. They proposed a differential attack based on deep learning against 15 rounds of Simon 32/64. Hou et al.[15] proposed a SAT-based algorithm to improve the neural distinguisher in 2021, demonstrating improved key recovery efficiency against cryptosystems with large block sizes. They further explored the accuracy of the neural distinguisher using different models of input differences[16], although its accuracy was lower than when using cipher pairs with the single ciphertext pair model. A best difference model $(0\times0, 0\times200)$ was used to construct a key recovery attack against the 11-round and 13-round versions of Simon 32/64. Zhu et al.[17] introduced the

[1]School of Software, Northwestern Polytechnical University, Xi'an 710000, China. [2]Information Security Research Center, CEPREI, Guangzhou 511370, China. [3]Department of cybersecurity, Shandong Future Network Research Institute, JiNan, China. [4]Institute of Cyberspace Security, Wuhan University, Wuhan 430072, China. ✉email: lucky_jiel@163.com

polyhedron neural distinguisher into key recovery attacks against Simon 32/64, improving both the number of attack rounds and the success rate. Wang et al.[18] used a greedy algorithm to select the best difference and Hyperopt to optimize the parameters and neural network. Neural distinguishers have been applied to analyze various cryptosystems such as TweGift-128, Simeck, GIFT-COFB, GIFT-64, and PRIDE[19–22]. In 2022, Baksi[23] trained the neural distinguisher using multi-layer perceptrons, single-difference, and multiple-difference models to achieve computational advantages. Bao et al.[24] proposed an enhanced difference-neural attack to recover the key for 13 rounds of Speck 32/64 and 16 rounds of Simon 32/64, achieving successful attacks on higher-round systems. In their work, they used simultaneous neutral bits and switching bits for adjoining differentials to enhance the neural distinguisher. In the same study, they used the inception model to improve both the attack round and accuracy. In[25], deep learning-based output prediction was combined with white-box and black-box attacks to explore deep learning-based cryptanalysis.

In contrast to these neural distinguishers, which use a single ciphertext pair as input, Chen et al.[26] proposed a neural distinguisher that uses multiple ciphertext pairs as input. They also introduced a neural-aided statistical attack to reduce attack complexity[27,28], combining deep learning with traditional statistical methods. In[29], ResNet was used to construct the neural distinguisher, enabling more efficient capture of features from multiple ciphertext pairs. The effectiveness of the features learned by the neural distinguisher was evaluated using false negative and false positive tests. Gohr et al.[30] optimized neural distinguishers for different cryptosystems, discussing the relationship between the accuracy of the neural distinguisher and the distribution of ciphertext differences. They proposed an automatic optimization strategy to achieve the best accuracy for different cryptosystems and found that the best input difference is not necessarily the optimal choice for a differential neural distinguisher. Specifically, they noted that improvements in the neural distinguisher using multiple ciphertext pairs at once, based on the work in[10], are marginal at best. Research on the application, optimization, and interpretability of neural distinguishers has been further explored by other researchers. Hou et al.[31] proposed aided attacks against the 13-round Simon 32/64 and 14-round Simon 48/96, achieving better key recovery with lower computational costs. In[32], Bao et al. introduced the freezing layer method to train distinguishers for high-round systems, finding that the neural distinguisher benefits from correlations between partial ciphertext values, differences in ciphertext pairs, and differences in intermediate states. Liu et al.[33] proposed a novel data generation model to improve neural distinguisher accuracy by extending one-dimensional data to two-dimensional data. The new data combines ciphertexts and ciphertexts obtained by decrypting one round with a random key, referred to as Multi-Round and Multi-Splicing Pairs (MRMSP) and Multi-Round and Multi-Splicing Differences (MRMSD). In 2024, Lu et al.[34] presented a new data format consisting of multiple ciphertext pairs and differences to achieve high accuracy and better attack rounds for the Simon and Simeck cryptosystems. The differences in the differential dataset are analyzed in[35] to obtain a better dataset. Additionally, the relationship between ciphertext pairs and their differences is explored in[36].

Motivation

Neural distinguisher-based key recovery attacks have been widely studied for various block ciphers and have achieved higher key recovery rates. However, there is still much work to be done, such as optimizing the input data model, improving the neural network, enhancing interpretability, and advancing the key recovery attack. In this work, we explore the construction of input data, the optimization of the neural network, and their performance in key recovery attacks.

Our contributions

This paper focuses on the accuracy of the neural distinguisher, which influences the success rate of key recovery attacks while addressing security aspects. The main contributions of this work are as follows:

(i) We design three novel neural networks based on ResNeXt and DenseNet, in which both the residual block and the convolutional layers are improved. Then derive rules of thumb for optimizing the neural network modules: increase the number of convolution layers in the residual block from 2 to 5. Start with a convolution kernel size of 1 and increase it by 2 for each successive convolution layer. Increase the convolution kernel size between different residual blocks by 2 with each step.

(ii) We combine the ciphertext pairs and their differences, the keys and the difference of the keys to the dataset. This approach highlights that the number of ciphertext pairs, the number of decrypted rounds, and the order in which these data are combined significantly affect accuracy. Finally, construct the dataset with highest accuracy by using the ciphertext pairs of different rounds, the differences of the ciphertext pairs of different rounds, and the differences of the keys used to decrypt the ciphertexts of different rounds.

(iii) A key recovery attack based on the proposed neural distinguisher is designed and implemented for the Speck 32/64 and Simon 32/64. This attack shows a 9.7% improvement in key recovery rate compared to the state-of-the-art methods for Speck 32/64.

The organization is as follows: The paper is organized as follows. Section 2 provides an introduction to deep learning-based differential cryptanalysis and introduces the Speck cipher. The proposed neural networks and the rules for optimizing them are described in Sect. 3. In Sect. 4, we design two efficient data models and explore the construction of the input dataset using different cipher materials and orders. In Sect. 5, the accuracy of the proposed neural distinguisher for Speck 32/64 and Simon 32/64 is tested. Additionally, we verify the key recovery rate of the proposed neural distinguisher using single ciphertext pairs as input. Finally, the work is concluded in Sect. 6.

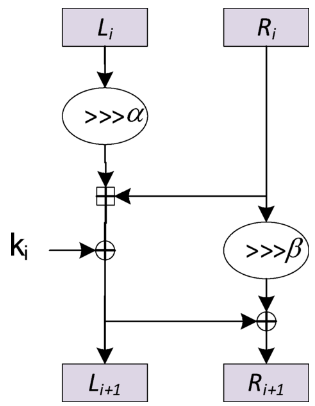

Fig. 1. The round function and key schedule algorithm of Speck 32/64.

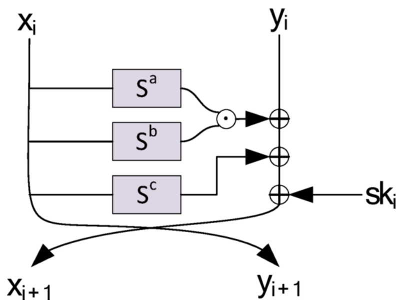

Fig. 2. The round function of Simon cipher.

Preliminaries

Let (x, x^*) denotes a plaintext pair with difference Δx. The corresponding ciphertext pair is (c, c^*), and the i-th round's ciphertext pair is (x_i, x_i^*). The notations used in this work are defined as follows:

Definition 1 (*Difference*). For any plaintext $x, x^* \in \{0,1\}^n$, the difference between x and x^* is defined as $\Delta x = x \oplus x^*$.

Definition 2 (*Difference of the ciphertext pair*). For any plaintext $x, x^* \in \{0,1\}^n$with a difference $\Delta x = x \oplus x^*$, their i-th round's ciphertext are x_i and x_i^*, respectively. Then $\Delta x_i = x_i \oplus x_i^*$ is the difference of the ciphertext pair.

Definition 3 (*Neutral bit*). Let $\{e_0, e_1, \ldots, e_i, \ldots, e_{n-1}\} \in F_2^n$ be the standard basis, where $i \geq 0$ is the index. Suppose that any plaintexts $x, x^* \in \{0,1\}^n$ have an input difference Δx, and their i-th round's ciphertext difference is Δx_i. If the difference of i-th round's ciphertext of $x + e_i$ and $x^* + e_i$ also equals Δx_i, then the i-th bit is the neutral bit of the difference $(\Delta x \to \Delta x_i)$.

The speck 32/64 cipher

The Speck cryptosystem is a lightweight block cipher with an ARX (Addition, Rotation, and XOR) construction. It is proposed by Beaulier et al.[11] for the NSA, aimed at achieving high efficiency for IoT devices. Different rotation constants and the number of rounds are designed for various block and key sizes. A general description of Speck is Speck B/A, where B denotes the block size and A denotes the key size. This work focuses on Speck 32/64 with 22 rounds. The round function is shown in Fig. 1.

As shown in Fig. 1, the plaintext is split into two subblocks (L_i, R_i) with each length of 16 bits, where i denotes the i-th round. k_i is the 16 bits key for the i-th round, generated by the round key schedule algorithm using mast key K. The round function follows a classical Feistel structure, combining XOR, addition modulo, and bitwise operations, which are denoted by $\oplus$, $\boxplus$ and $\odot$, respectively. The bitwise left and bitwise right rotations are denoted by <<< and >>>, respectively. In the round function of Speck 32/64, $\alpha = 7$ and $\beta = 3$.

The Simon 32/64 cipher

Simon[11] is a lightweight block cipher proposed by the NSA to meet the need for secure, flexible, and analyzable encryption. Its block sizes are 32, 48, 64, 96, and 128 bits, which can be expressed in units of word lengths as 16, 24, 32, 48, and 64, respectively. The round function of Simon is shown in Fig. 2.

As shown in Fig. 2, the key-dependent Simon $(2 \cdot n)/(l \cdot n)$ round function is the map $R_{sk_i}: GF(2)^n \times GF(2)^n \rightarrow GF(2)^n \times GF(2)^n$, defined by formula (1), where n is the block size in word length unit, and l is the number of the key word length.

$$R_{sk_i}(x_i, y_i) = (y_i \oplus f(x_i) \oplus sk_i, x_i) \tag{1}$$

Where $f(x_i) = \left(S^1 x_i \odot S^8 x_i\right) \oplus S^2 x_i$, $\odot$ is the bitwise AND operation, and $sk_i \in GF(2)^n$ is the round key.

Structure of the Gohr' neural network

The structure of the neural network used for differential analysis was first proposed by Gohr in[10]. Its network is shown in Fig. 3:

In[10], the ciphertext pair (x, x^*) is generated by encrypting two plaintext pair with a specified difference Δx, for example $\Delta x = (0 \times 0040/0000)$. The ciphertext pair (x, x^*) encrypted by Speck 32/64 is first split into two words. Then, it goes through the initial convolutional (IC) model, where a convolution with a filter size of 1 and 32 filters is applied. Batch normalization and a Rectified Linear Unit (ReLU) activation layer follow the convolutional layer. Next, 10 Convolutional Block (CB) models are applied, each consisting of two iterations of convolutional layers with a filter size of 3 and 32 filters, followed by batch normalization and ReLU activation. The input of each CB model is added to its output. Finally, the output of the last CB model is processed by the prediction head (PH) model, which consists of two iterations of a densely connected layer (using 64 neurons), batch normalization, and ReLU activation. The output of the PH model is processed by a densely connected layer in the output model, which uses a sigmoid activation function and a single neuron.

The differential analysis

Differential analysis is a commonly used method for data analysis, which extracts significant features and correlations between two or more data sets. It is a form of chosen plaintext attack. This method distinguishes ciphertext pairs of a given difference from random ciphertext pairs based on the probability propagation property of the plaintext pair with the given difference during encryption. Given an encryption function $E: F_2^{n_p} \times F_2^{n_k} \rightarrow F_2^{n_c}$, where n_p denotes the number of the plaintext bits, n_k denotes the number of the key bits, and n_c denotes the number of the ciphertext bits. Let the input of plaintext pair be (x, x^*), the difference of x and x^* is denoted by $\Delta x = x \oplus x^*$. The ciphertext-pair of i-th round is (x_i, x_i^*), and its difference is denoted by $\Delta x_i = x_i \oplus x_i^*$. After n rounds of encryption, we obtain a differential sequence $\Omega = (\Delta x, \Delta x_1, \cdots, \Delta x_n)$, which is called the differential path of the n-rounds of encryption. This path represents the differential propagation characteristics of the encryption. Let N_D be the number of the plaintext pair. The probability that all these plaintext pair have a differential path Ω is given by formula (2):

$$R_P(\Delta x, \Delta x_n) = \frac{N_D}{2^m} \tag{2}$$

where m is the length of the plaintext. The Differential Distribution Table (DDT) is widely used to analyze cryptosystems, listing the probability of each differential path.

In[10], Gohr constructed a neural distinguisher (ND) for Speck 32/64 reduced to 8 rounds and recovered the key for Speck 32/64 reduced to 11 rounds. First, Gohr extended the 7-round distinguisher to a 9-round distinguisher by prepending a two-round differential transition $\Delta x = \delta \rightarrow \Delta x_2 = (0x0040{,}0000)$, passing as desired with a probability of about 1/64. For example, $\Delta x = \delta = (0x0211{,}0x0a04)$. Then, the 9-round distinguisher was extended by another round with no additional cost, requiring encryptions of ciphertext pairs (x_0, x_0^*) that encrypt to the desired input difference δ after one round of Speck encryption. This creates a 10-round distinguisher, which can be easily achieved since no key addition happens in Speck before the first nonlinear operation. Finally, the ciphertext pair (x_{11}, x_{11}^*) is decrypted by a random key, and recovery the key by the 10-round distinguisher and Bayesian optimization.

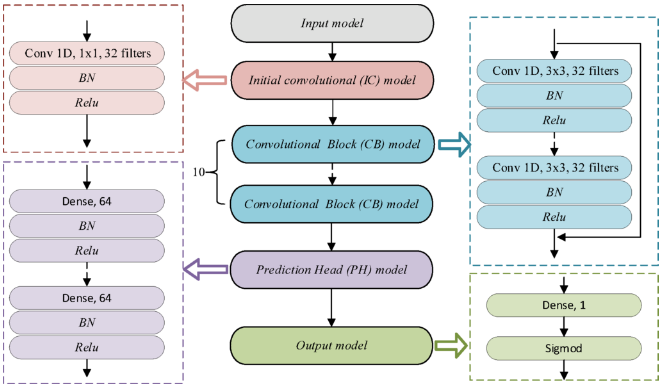

Fig. 3. The structure of Gohr's neural network.

Design of the neural network

Inspired by the pyramid convolutional structure and the residual network, we introduce multi-scale convolution and multi-character studies into the convolutional layer to propose a novel distinguisher (ND). Furthermore, different convolutional block models are designed to enhance the accuracy of the proposed ND. Additionally, three different convolutional block models are designed to use as the ablation schemes and validate the efficiencies of the proposed ND.

The distinguisher using pyramid convolutional structure

Since the pyramid convolutional structure can capture both global and local features, it is employed to optimize the convolutional module of Gohr's neural network and construct eleven different distinguishers. We test the accuracy of these distinguishers using different number of convolutional blocks N_{CB}, convolutional layers N_{CL}, first convolutional kernel size of each CB model N_{CK}, and the increasement of convolutional kernel size N_{step} in each CB block. The results of these distinguishers against 5 round-reduced Speck 32/64 cipher are shown and compared with that of Gohr's distinguisher in Table 1:

As shown in Table 1, the distinguisher with the highest accuracy is ND_G8, which has five CB models, each with five convolutional layers. The convolutional kernel sizes for the first CB model are 1, 3, 5, 7, and 9 for each convolutional layer, with the corresponding convolutional kernel size of each subsequent CB model increasing by a step of 2. The accuracy of all the improved distinguishers is higher than that of Gohr's, which implies that the pyramid convolutional structure can improve the accuracy of the ND.

From above, we derive the structure optimization criteria of neural discriminator network based on ResNet as follows:

Proposition 1 The accuracy of the neural distinguisher can be enhanced by increasing the number of convolutional layers in the residual block from 2 to 5.

Proposition 2 The accuracy of the neural distinguisher can be enhanced by increasing the convolutional kernel size in the residual block from 1, with a step of 2.

Proposition 3 The accuracy of the neural distinguisher can be enhanced by increasing the initial convolutional kernel size in different residual blocks with a step of 2.

New distinguisher base on multiple scale convolution and denser residual connection

Based on the above propositions, we introduce the denser residual connection in[37] and construct a novel CB model. Then, present a new neural distinguisher ND_1 with five proposed CB models, while the other models remain the same as Gohr's. The network structure is shown in Fig. 4.

In Fig. 4, ks denotes the initial convolutional kernel size, and i denotes the i-th convolutional block. The operation $\oplus$ denotes addition. In the new convolutional block, a denser residual connection is added from the input to all the convolutional layers. The n-th convolutional layer is defined by formula (3).

$$F^n(x) = F\left(\sum {}_{i=1}^{n-1} F^i(x) + x\right) \tag{3}$$

where x is the input of the n-th layer, $F^i(x)$ is the output of the i-th layer. The final output of the convolutional block model is $F^{N_{CL}}(x)$. Additionally, a skip connection is added from the output of each convolutional layer to the input of all subsequent convolutional layers, while the convolutional kernel size of the residual block increases by 2. These skip connections allow the use of features from lower layers, and the addition operation preserves the shape of the feature matrix, preventing the data dimension from exploding.

The ciphertext pairs are first reshaped into a two-dimensional array with 16 columns. Then, they are processed by a convolution operation with a kernel size of 1, followed by Batch Normalization and the ReLU function.

ND	N_{CB}	N_{CL}	N_{CK}	N_{step}	Accuracy
ND_G1	5	2	1,1	2	93.21%
ND_G2	5	2	1,3	2	93.12%
ND_G3	5	2	3,3,	2	93.19%
ND_G4	5	3	1,3,5	0	93.01%
ND_G5	5	4	1,3,5,7	0	93.03%
ND_G6	5	4	3,5,7,9	0	93.16%
ND_G7	5	4	3,5,7,9	2	93.24%
ND_G8	**5**	**5**	**1,3,5,7,9**	**2**	**93.25%**
ND_G9	5	5	3,5,7,9,11	2	93.08%
ND_G10	8	4	3,5,7,9	2	93.18%
ND_G11	10	4	3,5,7,9	2	93.16%
Gohr	10	2	3,3	0	92.95%

Table 1. The accuracy of different improvements of Gohr's neural network.

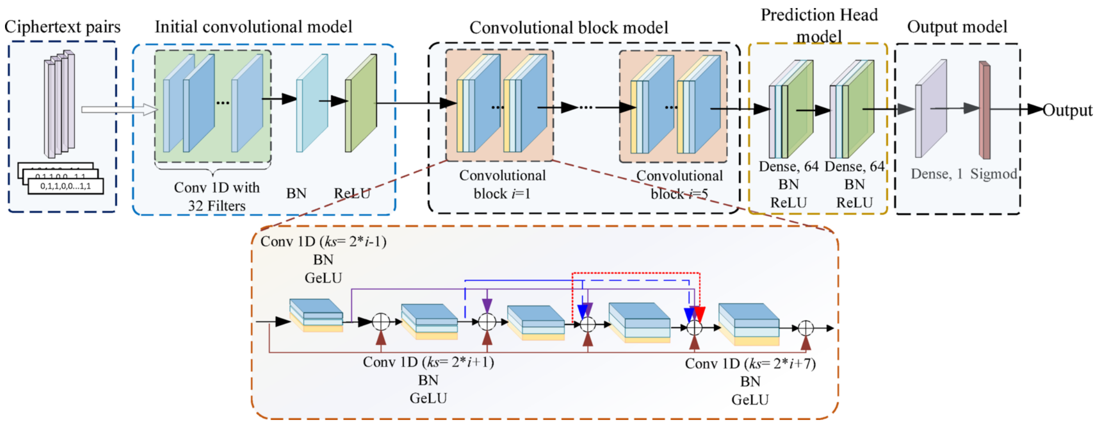

Fig. 4. The network structure of the proposed neural distinguisher.

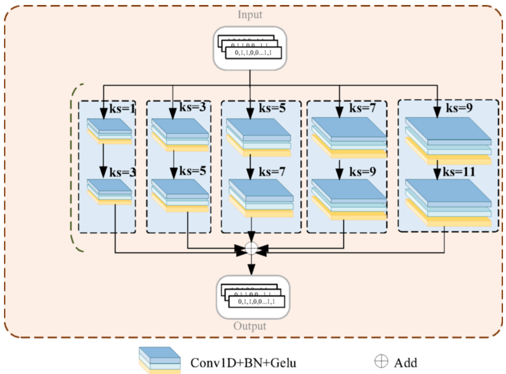

Fig. 5. The structure of the convolutional block model with parallel convolutional blocks.

In the convolutional block model, the data is processed through five convolutional blocks, each containing different convolutional layers. In each convolutional block, there are five convolutional layers with an increasing convolutional kernel size, incrementing by 2, enabling feature extraction at different scales. The output of the convolutional block model is processed by two dense blocks, each containing a dense layer of size 64, a Batch Normalization operation, and a ReLU function. Finally, the features are mapped into the target space by a dense layer of size 1 and passed through the Sigmoid function. By using the multi-scale convolution operation in the convolutional block model, more features of the ciphertext pairs can be captured, allowing the new neural distinguisher to achieve higher accuracy.

Design of the ablation schemes

In the proposed distinguisher ND_1, the input of each convolutional layer is the output of its previous convolutional layer, except for the first convolutional layer. To verify whether this design may lead to the loss of some features from the original input, we consider using convolutional layers with different kernel sizes simultaneously to process the original input. We replace the CB model of ND_1 with five parallel CB blocks, each containing different convolutional layers, to obtain a new neural distinguisher ND_2. The structure of the CB model of ND_2 is shown in Fig. 5.

Here, the parameters ks represents the initial convolutional kernel size. As shown in Fig. 5, there are five convolutional blocks, each consisting of two identical convolutional layers. The convolutional operation of each identical layer can be defined as a function $F(x)$. The convolutional kernel size of the convolutional layers increases with a step of 2. Multiple groups of convolutional layers with different kernel sizes can study features in parallel, thereby enhancing the model's expressive power. This approach also reduces the number of parameters and computational cost.

Additionally, the initial convolutional model of ND_1 uses a single convolutional layer with a kernel size of 1 to capture the features of the ciphertext pairs. To verify whether multiple convolutional layers with different kernel sizes can capture more nonlinear features that are widely present in ciphertext pairs, we replace the initial convolutional model of ND_1 with a new initial convolutional model containing multiple convolutional layers with different kernel sizes. The resulting neural distinguisher is denoted as ND_3. The structure of the new initial convolutional model is shown in Fig. 6.

In Fig. 6, the input is processed simultaneously by three convolutional layers at the first depth, each with a kernel size of 1 and denoted as $F^{1,1}(x)$, $F^{1,2}(x)$ and $F^{1,3}(x)$, respectively. Then, the outputs of $F^{1,1}(x)$ and $F^{1,3}(x)$ are concatenated and used as the input for the first convolutional layer at the second depth, defined as $F^{2,1}(x) = F(F^{1,1}(x), F^{1,3}(x))$. The second convolutional layer at the second depth is defined as $F^{2,2}(x) = F(F^{1,2}(x), F^{1,3}(x))$. Following the same method, the convolutional layer at the third depth is defined as $F^{3,1}(x) = F(F^{2,1}(x), F^{2,2}(x))$. Finally, the three outputs are added together to generate the output of the initial convolutional model.

Best parameters and network structure for the proposed neural distinguisher

To verify the effectiveness of the proposed neural distinguisher, the accuracy of the neural distinguishers ND_1, ND_2, and ND_3 is tested. The simulation environment is set up as follows:

Hyperparameters

We set the number of epochs to 200 and the batch size to 5000. The default parameters of the Adam algorithm in Keras[38] are used to optimize the mean squared error (MSE) loss, with a small penalty based on $L2$ weights regularization, and a regularization parameter $c = 10^{-5}$. The learning rate for epoch i is defined as formula (4).

$$l_i = \alpha + \frac{(n-i)\ mod\,(n+1)}{n} \cdot (\beta - \alpha) \tag{4}$$

where $\alpha = 10^{-4}$, $\beta = 2 \cdot 10^{-3}$, and $n = 9$.

Data generation

The random number generator in Linux, using a fixed random seed, is used to generate the keys, training dataset, and test dataset. The size of the training set is 10^7, while the size of the test set is 10^6. Half of the dataset is generated by encrypting the plaintext pairs with an input difference $\Delta x = (0x0040, 0x0000)$, which is labeled by 1. The other half of the dataset is generated by encrypting random plaintext pairs, which are labeled as 0.

Considering the improvements to Gohr's NDs in Table 1, which achieve higher accuracy when $N_{CK} = (3,5,7,9)$ and $N_{CK} = (1,3,5,7,9)$, we test the accuracy of ND_1 and ND_2 against 5 round-reduced Speck 32/64 for different N_{CK} in Table 2:

From Table 3, we can see that the accuracy of the proposed distinguishers ND_1 is approximately equal to the ablation scheme ND_2. Therefore, applying parallel convolutional layers with different kernel sizes to the original input has a limited effect on the neural distinguisher. Furthermore, ND_2 requires more training time and computational resources, as it requires copying multiple inputs to the CB model and uses more convolutional layers. Therefore, ND_1 is more suitable for large input datasets. Although the accuracy of another ablation scheme ND_3 is higher than that of the other distinguishers for both 5-round and 6-round reduced Speck 32/64, it is lower than the distinguishers in[10,30] for the 7-round reduced Speck. Moreover, its accuracy is significantly lower than that of ND_1, suggests that multi-scale convolution in the initial convolutional model fails to capture additional nonlinear features effectively. The accuracy of ND_1 is higher than that of the traditional distinguisher and the ND models in[10,29], and[30]. The advantage ranges from 0.2 to 0.5%, which is significant according to Gohr's evaluation criteria for neural distinguishers.

Additionally, the training time, inference speed, memory usage, and number of parameters of the different neural distinguishers are tested on an Intel(R) Xeon(R) Silver 4216 CPU @ 2.10 GHz platform with an RTX 3090 GPU. The results are shown in Table 4.

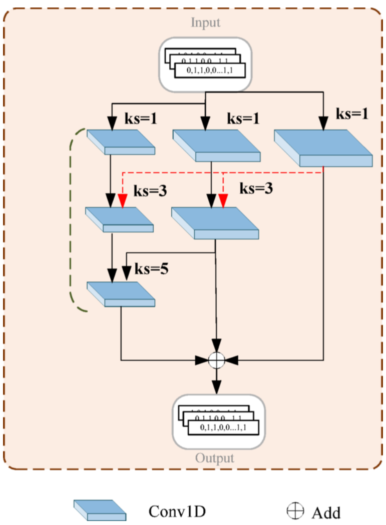

Fig. 6. The structure of the initial convolutional model with multi-scale layers.

Round	ND	Accuracy
5	ND_1 with $N_{CK}=(3{,}5,7{,}9)$	93.29%
	ND_2 with $N_{CK}=(3{,}5,7{,}9)$	93.38%
	ND_1 with $N_{CK}=(1{,}3,5{,}7,9)$	93.35%
	ND_2 with $N_{CK}=(1{,}3,5{,}7,9)$	93.36%
	Gohr's	92.95%

Table 2. The accuracy of the NDs using improved CB model with different N_{CK} From Table 2, we can see that the accuracy of ND_1 and ND_2 obtains the maximum advantage of 0.34–0.43% compared to Gohr's when $N_{CK}=(1{,}3,5{,}7,9)$. Therefore, we adopt $N_{CK}=(1{,}3,5{,}7,9)$ for the proposed ND_1 in the following work. Furthermore, we validate the efficiency of ND_1 and the ablation schemes, and compare them with other distinguishers in Table 3:

ND	Accuracy		
	Round 5	Round 6	Round 7
D_5^* [10]	91.1%	75.8%	59.1%
CSYY [29]	92.6%	78.4%	60.7%
Goh19* [10]	92.9%	78.8%	61.6%
Goh22 [30]	92.9%	78.8%	61.7%
$ND1$	93.35%	79.06%	61.85%
$ND2$	93.36%	79.1%	61.87%
$ND3$	93.1%	79%	61.3%

Table 3. Comparison of accuracy of different neural distinguishers. *D denotes the traditional distinguisher. *Ghor denotes the neural distinguisher that uses knowledge distillation for the 7 round-reduced Speck 32/64.

ND	Training time of single epoch (s)	Inference speed (ms)	Memory usage (KB)	Number of parameters
Goh19 [10] (baseline)	88	25	6,181,116	100,897
ND_1	117	30	6,223,800	270,337
ND_2	205	36	6,534,868	503,137
ND_3	112	27	6,086,744	114,817

Table 4. Comparison of the efficiency of the proposed neural distinguishers.

The number of the convolutional layers of the residual block equals to 5	The convolutional kernel of the residual block increases with a step of 2	The convolutional kernel in different residual blocks increases with a step of 2	Using multiple scale convolution and denser residual connection	Accuracy
×	×	×	×	92.9%
√	×	×	×	93.07%
√	√	×	×	93.18%
√	√	√	×	93.25%
√	√	√	√	93.36%

Table 5. The accuracy of the ND_1 with different ablation experiments

Table 4 shows that, the proposed ND_1 requires less time than ND_2. Although the efficiency of ND_1 is lower than ND_3, its accuracy is higher. Since accuracy is more critical for key recovery than slightly increased resource consumption, the proposed ND_1 represents the best compromise between accuracy and efficiency.

To verify the validity of the above optimization criteria (Propositions 1–3) for ND_1, we conduct an ablation experiment with different combinations of the criteria. The details are shown in Table 5.

Table 5 demonstrates that the combination of different improved components in ND_1 contributes to a 0.46% improvement in accuracy.

Thus, we can conclude that the three propositions in Sect. 3.1 are applicable to the proposed neural distinguisher ND_1, ND_2 and ND_3. The denser residual connection and the multi-scale convolutional layers in the convolutional block module can significantly improve the accuracy of the neural distinguisher. Among them, ND_1 achieves the best balance between efficiency and accuracy compared to ND_2 and ND_3.

Mathematical analysis of feature extraction improvement

The convolutional kernel size in[10] remains constant at 3, which results in a limited and small receptive field. In this work, the convolutional kernel size within the residual block and between residual blocks increases by a step of 2, leading to a richer convolutional receptive field and an extended feature range. The computation of the receptive field is defined by formula (5):

$$RF_{i+1} = RF_i + (ks-1) \times \ S_i \tag{5}$$

where RF_{i+1} denotes the receptive field of the $(i+1)$-th layer, ks represents the convolutional kernel size, and S_i denotes the product of the all the steps from the first layer to the i-th layer. Since the convolutional step of both ND_1 and the neural distinguisher in[10] is 1, the receptive field can be denoted by $RF_{i+1} = RF_i + (ks-1)$. Therefore, the receptive field of ND_1 becomes larger than that of the neural distinguisher in[10] from the third convolutional layer. Furthermore, the receptive field of ND_1 increases at each convolutional layer and convolutional block, allowing it to capture multi-scale features. Additionally, based on the definition of the neural network in formula (3), we analyze the process of each convolutional layer in the proposed ND_1 and Gohr's neural distinguisher in[10]. Let x_l denotes the input of the neural distinguisher, l denotes the l-th convolutional block. The output of each convolutional layer is presented in Table 6:

Here, $Conv\,1D\,(a,b,c)$ denote a 1D convolution operation using convolutional layer weight a, convolutional kernel size b, and input c. W_i^l denotes the weight of the i-th convolutional layer in the l-th convolutional block. Table 6 shows that the final output of ND_1 integrates all the features captured by different convolutional layers, whereas the final output of the neural distinguisher in[10] only contains the features captured by the current convolutional layer. Therefore, the modifications introduced in this work significantly enhance the performance of the distinguisher.

Constructing of dataset using multiple ciphertext pairs

Besides the improvement of the neural network structure, the construction of the dataset is another important way to improve the accuracy of the neural distinguisher. In this section, we propose two methods to construct the dataset using different rounds of ciphertext pairs, decryption keys, and ciphertext pair differences. Then obtain two optimal combinations of these elements and test their accuracy.

Two novel models of dataset construction

Since the input data can provide different features for the ND, we construct two new datasets by combining ciphertext pairs, decryption keys, and ciphertext pair differences from different decryption rounds. These datasets are denoted as follows: multi-round multi-splicing ciphertext-pair and keys (MRMSCPK), and multi-round multi-splicing ciphertext-pairs and differences (MRMSCPD).

In MRMSCPK, the ciphertext pair (x_i, x_i^*) is decrypted by a random key to obtain the ciphertext pair (x_{i-1}, x_{i-1}^*) and (x_{i-j}, x_{i-j}^*) is obtained by decrypting j times with j different keys. The dataset is denoted by $\{(x_i,x_i^*)_1, \cdots (x_i,x_i^*)_{N_d}, (x_{i-1},x_{i-1}^*)_1, \ \cdots, (x_{i-1},x_{i-1}^*)_{N_d}, key\}$, where $(x_i,x_i^*)_l$ denotes the l-th ciphertext pair of i-round's encryption, $key=\{key_1,\cdots,key_j\}$, and N_d is the number of the ciphertext pairs. The structure is shown in Fig. 7.

Here, N_D is the number of ciphertext pairs. x_{i-j} denotes the ciphertext obtained by decrypting a i-round's ciphertext j times. The size of the dataset is $(2m\cdot\ N_D\cdot\ j+j)$.

In MRMSCPD, the dataset consists of the ciphertext pairs and the difference of the ciphertext pairs, which is denoted as $\{(x_i,x_i^*)_1, (\varDelta\, x_i)_1, \cdots, (x_i,x_i^*)_{N_d}, (\varDelta\, x_i)_{N_d}, \cdots, \left(x_{i-j},x_{i-j}^*\right)_1, \ (\varDelta\, x_{i-j})_1, \cdots, \left(x_{i-j},x_{i-j}^*\right)_{N_d}, (\varDelta\, x_{i-j})_{N_d}\}$. Here, $(x_i,x_i^*)_l$ denotes the l-th ciphertext pair of i-round's encryption, and $(\varDelta\, x_i)_l$ denotes the difference of the l-th ciphertext pair. The detail is shown in Fig. 8.

Index of the convolutional layer	Output of the convolutional layer	
	ND_1	ND in [10]
1	$Conv_1 = Conv\,1D\left(W_1^l, 2L-1, x_l\right)$ $R_1 = x_l + Conv_1$	$Conv_1 = Conv\,1D\left(W_1^l, 3, x_l\right)$
2	$Conv_2 = Conv\,1D\left(W_2^l, 2L+1, R_1\right)$ $R_2 = x_l + Conv_1 + Conv_2$	$Conv_2 = Conv\,1D\left(W_2^l, 3, Conv_1\right)$
3	$Conv_3 = Conv\,1D\left(W_3^l, 2L+3, R_2\right)$ $R_3 = x_l + Conv_1 + Conv_2 + Conv_3$	×
4	$Conv_4 = Conv\,1D\left(W_4^l, 2L+4, R_3\right)$ $R_4 = x_l + Conv_1 + Conv_2 + Conv_3 + Conv_4$	×
5	$Conv_5 = Conv\,1D\left(W_5^l, 2L+7, R_4\right)$	×
Final output	$y_{l+1} = x_l + Conv_5$	$y_{l+1} = x_l + Conv_2$

Table 6. The output of each convolutional layer for different neural distinguishers.

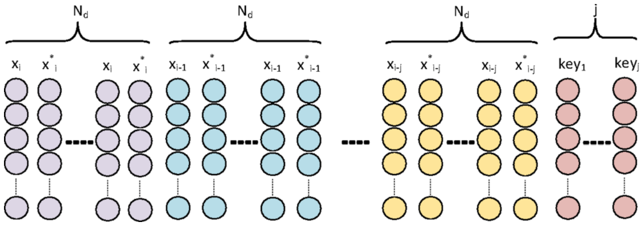

Fig. 7. The data set structure of MRMSCPK.

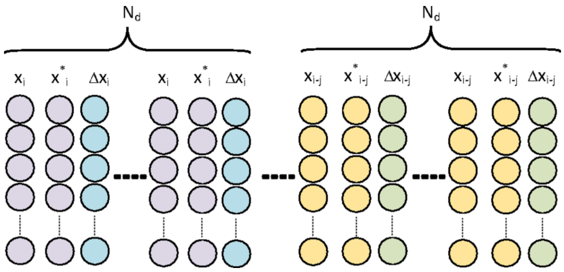

Fig. 8. The data set structure of MRMSCPD.

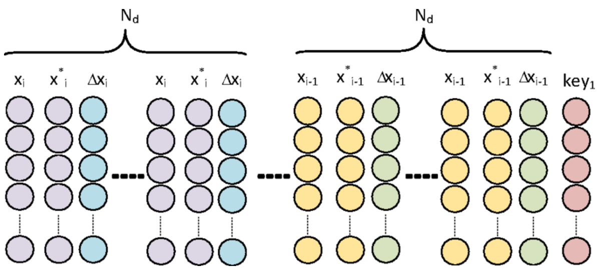

Fig. 9. The data set structure of DRMSCPDK.

Construction of dataset with different decryption rounds

Although the decrypted ciphertext pairs are useful for capturing features, the error propagation introduced by decryption with random keys may mislead the neural distinguisher. Therefore, we explore efficient decryption rounds to construct the datasets for MRMSCPK and MRMSCPD, where the decryption rounds j are varied from 1 to 2. When $j = 1$, the datasets are denoted by DRMSCPK and DRMSCPD, respectively. We use the ciphertext pairs and their differences from the i-th and $(i-1)$-th rounds, along with the keys used to decrypt the ciphertext of the i-th round, to construct double-round multiple splicing ciphertext pairs with differences and keys (denoted as DRMSCPDK). The structure of the DRMSCPDK is shown in Fig. 9.

For $j = 2$, the ciphertext pair (x_i, x_i^*) is decrypted to (x_{i-1}, x_{i-1}^*) and (x_{i-2}, x_{i-2}^*) using the same random key. All these ciphertext pairs and their differences are used to construct triple-round multiple splicing ciphertext pairs and differences (denoted as TRMSCPD), as shown in Fig. 10.

Furthermore, we use two different keys to decrypt (x_i, x_i^*) and add the differences of the keys at the end of the data in Fig. 10 which is denoted as TRMSCPDKD (triple-round multiple splicing ciphertext pairs, differences, and key differences).

Since the dataset is reshaped to a $4 \times \frac{B}{2}$ matrix for each ciphertext pair and input into the initial convolutional layer, the order of the ciphertext pairs and their differences may affect the accuracy. Therefore, we change the order of DRMSCPDK to

$$\left\{(x_i, x_i^*)_1, \cdots (x_i, x_i^*)_{N_d}, \cdots (x_{i-1}, x_{i-1}^*)_1, \ \cdots, (x_{i-1}, x_{i-1}^*)_{N_d}, \ (\Delta\, x_i)_1, \cdots, (\Delta\, x_i)_{N_d}, (\Delta\, x_{i-1})_1, \cdots, (\Delta\, x_{i-1})_{N_d}, key\right\}$$

. This dataset is denoted as DRMSCPDK* and shown in Fig. 11.

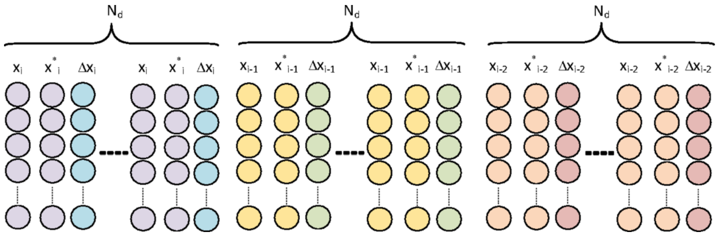

Fig. 10. The data set structure of TRMSCPD.

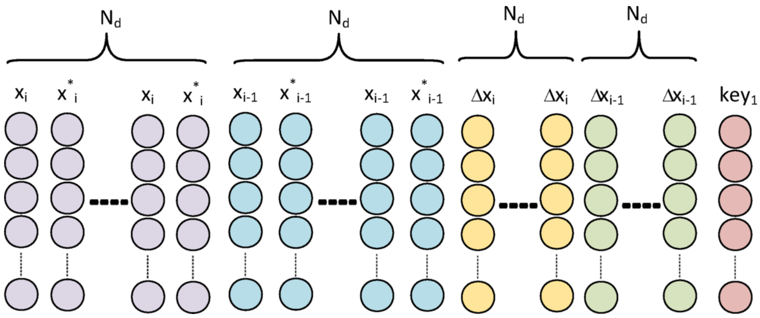

Fig. 11. The data set structure of DRMSCPDK*.

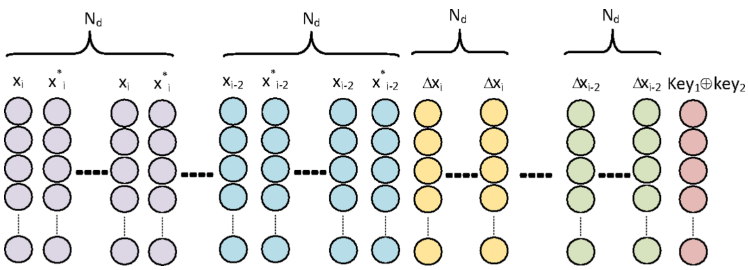

Fig. 12. The data set structure of TRMSCPDAK*.

Then, we change the order of TRMSCPDKD to

$$\left\{\begin{array}{l}(x_i,x_i^*)_1,\cdots(x_i,x_i^*)_{N_d},\cdots,(x_{i-1},x_{i-1}^*)_1,\cdots(x_{i-1},x_{i-1}^*)_{N_d},\\ \cdots,(x_{i-2},x_{i-2}^*)_{N_d},(\Delta x_i)_1,\cdots,(\Delta x_i)_{N_d},(\Delta x_{i-1})_1,\\ \cdots,(\Delta x_{i-1})_{N_d},(\Delta x_{i-2})_1,\cdots,(\Delta x_{i-2})_{N_d},key_1\oplus key_2\end{array}\right\}$$

. This is denoted as TRMSCPDKD* and shown in Fig. 12.

Test of accuracy and key recovery

To validate the efficiency of the different datasets, we train the neural distinguisher ND_1 using the proposed datasets and compare the accuracy of the different neural distinguishers. Additionally, we test key recovery using the proposed ND_1 for Speck 32/64 and Simon 32/64. The details are as follows.

Testing the proposed distinguisher with a new dataset format for speck 32/64

Since the neural distinguisher ND_1 achieves the best accuracy and training speed for a single ciphertext pair, we fine-tune the structure and reduce the number of convolutional layers in each convolutional block to 4 for handling multiple ciphertext pairs. The size of the training dataset and prediction dataset is 10^6 and 10^5, respectively. The epoch is set to 100, and the batch size is set to 1000. The learning rate of epoch i is defined as $l_i=\alpha+\frac{(n-i)\ mod\ (n+1)}{n}\cdot(\beta-\alpha)$, where, $\alpha=10^{-4}$, $\beta=2\cdot 10^{-3}$, and $n=9$. The number of ciphertext pairs is set to $N_D=32$, as used in[33]. The accuracy of the proposed ND_1 for Speck 32/64 is test and compared in Table 7.

From Table 7, we can see that the accuracy of all the proposed datasets using ND_1 is higher that of the neural distinguisher in[33], where the accuracy in[33] is the highest so far. The datasets DRMSCPDK* and TRMSCPDKD* achieve the highest accuracy for 8-round-reduced Speck 32/64 and 7-round-reduced Speck 32/64, respectively.

ND	Accuracy	
	Round 7	Round 8
MRMSP [33]	89.16%	57.74%
MRMSD [33]	94.11%	65.02%
DRMSCPDK	96.24%	65.35%
TRMSCPD (Proposed)	96.21%	65.09%
TRMSCPDKD (Proposed)	96.23%	65.27%
DRMSCPDK* (Proposed)	96.21%	65.71%
TRMSCPDKD* (Proposed)	96.27%	65.5%

Table 7. Comparison of the accuracy of different neural distinguisher for speck 32/64.

Round	ND	Number of ciphertext pairs n_{cp}							
		2	4	8	16	32	64	128	256
7	ND of [33]	63.33%	68.51%	75.82%	84.48%	94.11%	95.51%	98.12%	98.62%
	CSYY [29]	64.99%	69.17%	72.93%	71.34%	69.52%	58.46%	59.91%	×
	DRMSCPDK*/ $ND1$ (Ours)	64.62%	70.00%	79.26%	88.92%	96.21%	**99.01%**	**99.39%**	**99.86%**
	TRMSCPDKD*/ $ND1$ (Ours)	64.24%	70.78%	79.29%	88.84%	96.27%	**99.03%**	**99.67%**	**99.86%**
8	ND of [33]	52.96%	54.45%	56.91%	56.91%	65.02%	55.32%	52.79%	57.34%
	CSYY [29]	×	×	×	×	×	×	×	×
	DRMSCPDK*/ $ND1$ (Ours)	×	54.53%	57.45%	61.11%	65.71%	69.52%	**74.30%**	**71.58%**
	TRMSCPDKD*/ $ND1$ (Ours)	51.93%	53.78%	57.03%	57.03%	65.5%	69.41%	**73.13%**	**72.40%**
9	ND of [33]	×	×	×	×	×	50.46%	50.54%	50.65%
	CSYY [29]	×	×	×	×	×	×	×	×
	DRMSCPDK*/ $ND1$ (Ours)	×	×	×	×	×	**50.52%**	**51.01%**	**50.91%**
	TRMSCPDKD*/ $ND1$ (Ours)	×	×	×	×	×	50.64%	50.79%	50.67%

Table 8. The accuracy of different neural distinguishers using different numbers of ciphertext pairs for speck 32/64.

Both datasets show an advantage of 2.1–2.16% for 7-round-reduced Speck 32/64, and 0.48–0.69% for 8-round-reduced Speck 32/64. The improvement in accuracy is significant.

We analyzed the reason that these particular data organizations capture more distinguishing features from the perspective of theoretical cryptography. As shown in Fig. 2, the $(i+1)$-th ciphertext can be obtained by formula (6).

$$\begin{cases} C_L^{i+1} = ((C_L^i >>> 7) \boxplus C_R^i) \oplus k^i \\ C_R^{i+1} = ((C_R^i <<< 2)) \oplus C_L^{i+1} \end{cases} \tag{6}$$

That is, the $(i+1)$-th ciphertext contains information about the bit shift and XOR operations between the i-th ciphertext and the i-th key. The difference of the ciphertext pair can be defined by formula (7):

$$\begin{cases} C_{L0}^{i+1} \oplus C_{L1}^{i+1} = (((C_{L0}^i >>> 7) \boxplus C_{R0}^i) \oplus k^i) \oplus (((C_{L1}^i >>> 7) \boxplus C_{R1}^i) \oplus k^i) \\ C_{R0}^{i+1} \oplus C_{R1}^{i+1} = ((C_{R0}^i <<< 2) \oplus C_L^{i+1}) \oplus ((C_{R1}^i <<< 2) \oplus C_L^{i+1} \end{cases} \tag{7}$$

We have $C_{L0}^{i+1} \oplus C_{L1}^{i+1} = ((C_{L0}^i >>> 7) \boxplus C_{R0}^i) \oplus ((C_{L1}^i >>> 7) \boxplus C_{R1}^i)$ and $C_{R0}^{i+1} \oplus C_{R1}^{i+1} = \left(C_{R0}^i <<< 2\right) \oplus \left(C_{R1}^i <<< 2\right) = \left(C_{R0}^i \oplus C_{R1}^i\right) <<< 2$. In the left-hand side of the difference equation, the i-th key is eliminated. While the right-hand side contains only the bit shift operation of $\left(C_{R0}^i \oplus C_{R1}^i\right)$. Therefore, the difference of the ciphertext pair removes the confusion introduced by the XOR operation and the i-th key, making the ciphertext features more distinguishable.

In this section, keys are added to the dataset DRMSCPDK*, and key differences are included in the dataset TRMSCPDAK*. This enables the neural distinguisher to study them independently, allowing it to extract more features. These theoretical analyses also demonstrate that the proposed datasets help capture more distinguishing features.

Additionally, we test the accuracy of the proposed neural distinguisher using ND_1 with different number of ciphertext pairs n_{cp}, which help us explore the optimal number of ciphertext pairs. The results are shown in Table 8.

In Table 8, the symbol "×" denotes that the accuracy of the neural distinguisher is smaller than 50.5%, which indicates that it cannot make an efficient distinction. The accuracy of the proposed scheme is higher than that of[29] and [33] for all round-reduced Speck32/64 variants. The accuracy of the proposed neural distinguisher using the DRMSCPDK* dataset and the $ND1$ network is 3.5% higher than that of[29] when $n_{cp} = 64$. Its accuracy reaches 99.86% when $n_{cp} = 256$, which is 1.24% higher than[29]. For the 9-round-reduced Speck32/64, the accuracy of the proposed neural distinguisher is 51.01% when $n_{cp} = 128$, which is 0.47% higher than[29] and can make an efficient distinguish. Additionally, the accuracy of the neural distinguisher increases as n_{cp} increases, reaching its highest accuracy when $n_{cp} = 128$. This trend is particularly evident for 8- and 9-round-reduced Speck 32/64.

Testing the proposed distinguisher with a new dataset format for Simon 32/64

To validate the generalization of the proposed neural distinguisher and dataset, we test the accuracy of the proposed ND_1 with the dataset TRMSCPDKD* for different round-reduced Simon 32/64 ciphers. The number of the ciphertext pairs is set to 32 to compare with other literatures. The results are shown in Table 9.

In Table 9, the neural distinguishers proposed in[33] and our paper can distinguish ciphertext pairs with fixed input differences from those with random input differences with a probability higher than 60%. The accuracy of the proposed ND_1 with the TRMSCPDKD* dataset is 0.32%, 1.83%, and 0.44% higher than that of the neural distinguisher in[33] for 9, 10, and 11 round-reduced Simon 32/64 ciphers, respectively.

Therefore, the proposed neural distinguisher demonstrates a significant advantage over other neural distinguishers. Additionally, it achieves high accuracy for both the Speck 32/64 and Simon 32/64 ciphers when using the TRMSCPDKD* dataset. We also evaluate the accuracy of the proposed neural distinguisher on a 4-round reduced GIFT-64 cipher, achieving 97% accuracy. This suggests that the proposed neural distinguisher exhibits strong generalization capabilities for lightweight ciphers, which is consistent with the findings of other studies, such as[29] and [31].

Key recovery attack based on the neural distinguisher

The key recovery attack on 11-round Speck32/64 is proposed in[11], which combines the 2-round classical differential $(0x0211{,}0x0a04) \rightarrow (0x0040{,}0x0000)$ with the 7-round and 6-round neural distinguishers. Since there is no key addition operation before the first nonlinear operation in Speck 32/64, this method can be extended to 10 rounds for the 7-round distinguisher. All the 10-round ciphertext pairs $(x_{10},\ x^*_{10})_i$ are obtained by decrypting $(x_{11},\ x^*_{11})_i$ with all possible keys $k_{10,j}$, where $j \in \{1{,}2{,}\cdots,2^{16}-1\}$. All $(x_{10},\ x^*_{10})_i$ are input into the 7-round distinguisher to obtain a score $S_{k_{10,j}}$ for each possible key, as defined as formula (8):

$$S_{k_{10,j}} = \sum{}_{i=1}^{N} \log_2 \left(\frac{Z_i^{k_{10,j}}}{1 - Z_i^{k_{10,j}}} \right) \tag{8}$$

where N is the number of the ciphertext pairs, $Z_i^{k_{10,j}}$ is the score of $k_{10,j}$ used to decrypt the ciphertext pair $(x_{11},\ x^*_{11})_i$. When $S_{k_{10,j}}$ exceeds the threshold C_i, $k_{10,j}$ is returned as a candidate key. The same operation is applied to obtain $k_{9,j}$. Since this method is probabilistic, it may not return the correct keys when only one round of trial decryption is performed. Bayesian optimization is used to reduce the number of trial decryptions and improve the probability of finding the correct keys. Additionally, the upper confidence bound (UCB) is employed to prioritize the ciphertext structures with the highest scores during the last-key search process, thus reducing the time spent searching for less promising ciphertext structures. It can be calculated by formula (9):

$$S^i_{cp} = w^i_{max} + \alpha \cdot \sqrt{\log_2(j)/ns_i} \tag{9}$$

Here, w^i_{max} is the highest distinguisher score obtained so far for the i-th ciphertext structure, ns_i is the number of previous iterations in which the i-th ciphertext structure has been selected, j is the number of the current iteration, and $\alpha = \sqrt{N}$, where N is the total number of ciphertext structure available.

Key recovery test for speck 32/64

In this work, the key recovery attack is implemented through the following four steps:

Step 1: Generate the incorrect key response profile for our neural distinguisher ND_1 by encrypting 3000 plaintext pairs (x_0, x^*_0) for each key difference $\delta \in \{1{,}2{,}\cdots,2^{16}-1\}$ over 11 rounds to obtain the ciphertext pairs (x_{11}, x^*_{11}).

Step 2: Decrypt the ciphertext pairs using $E^{-1}_{k+\delta}(x_{11})$ and $E^{-1}_{k+\delta}(x^*_{11})$, then distinguish them using the proposed ND_1 to compute the empirical mean μ_δ and standard deviation σ_δ of these trials. Here, k is the final subkey of each encryption operation.

ND	Accuracy		
	Round 9	**Round 10**	**Round 11**
ND of [20]	82.27%	61.09%	×
MRMSP [33]	96.30%	78.72%	56.16%
MRMSD [33]	99.08%	83.02%	60.81%
TRMSCPDKD*/ $ND1$ (Ours)	99.4%	84.85%	61.25%

Table 9. Comparison of the accuracy of different neural distinguisher for Simon 32/64.

Step 3: Randomly select 100 plaintext pairs with an input difference of $(0x11, 0xa04)$ and decrypt them using all-zero key. Then, extend each of these by modifying the neutral bits {20,21,22,14,15,23} to create 64 plaintext pairs, and calculate the 11-round ciphertext pairs $(x_{11}, x_{11}^{*})_i$ of each.

Step 4: Use UCB method and the optimized Bayesian approach to recover the key of Speck 32/64.

The details of the key recovery process for the 11-round case are shown in Algorithm 1:

Input: $ND2$, $X = \{X_1, X_2, \cdots, X_N\}$ where $X_i = \{(x_{11}, x_{11}^{*})_1, \cdots, (x_{11}, x_{11}^{*})_m\}$, μ_δ, σ_δ, N_{ucb}, N_{kc}, n_k, c_1, c_2

Output: key

for i=1 to N_{ucb}
  X_i with max S_{cp}^{i} is chosen
  $key = \{k_1, k_2, \ldots, k_{n_k}\} \leftarrow$ choose at random without replacement from the set of all round key candidates.
  $L \leftarrow \{\}$
  for $iter$=1 to N_{kc}
    $P_{i,k_a} \leftarrow E_{k_a}^{-1}(X_i)$ for all $k_a \in key$, $a \in \{1,2,\cdots,n\}$ and $i \in \{1,2,\cdots,m\}$
    $v_{i,k_a} \leftarrow ND(P_{i,k_a})$ for all $k_a \in key$ and $i \in \{1,2,\cdots,m\}$
    $w_{i,k_a} \leftarrow \log_2(v_{i,k_a}/(1 - v_{i,k_a}))$ for all $k_a \in key$ and $i \in \{1,2,\cdots,m\}$
    $w_k \leftarrow \sum_{i=1}^{m} w_{i,k_a}$ for all $k_a \in key$ and $i \in \{1,2,\cdots,m\}$
    $L \leftarrow L \,||\, (k_a, w_k)$ for $k_a \in key$
    $m_k \leftarrow \sum_{i=1}^{m} v_{i,k_a}/m$ for all $k_a \in key$
    $\lambda_{k_a} \leftarrow \sum_{a=1}^{n_k}((m_{k_a} - \mu_{k_a+k})^2/\sigma_{k_a+k}^2)$ for $k \in \{0,1,\cdots,2^{16}-1\}$
    $S_{i,k} \leftarrow Sort(\lambda)$
  end for
  for a=1 to n
    if it is the first round of decryption and $S_{i,k_a} > c_1$ then
      return k_a and do the next round's key search
    else if it is the second round of decryption and $S_{i,k_a} > c_2$ then
      return k_a
    end if
  end for
end for

Algorithm 1. Key recovery process.

In Algorithm 1, the variable N_{ucb} denotes the iteration number of the UCB, N_{kc} denotes the iteration number of the Bayesian-based key search, n_k denotes the number of candidate keys, and c_1 denotes the threshold of the key's score. The function $E_{k_a}^{-1}(X_i)$ denotes the decryption of the ciphertext pair C_i by key k_a, and $ND(P_{i,k_a})$ denotes distinguishing the plaintext pair P_{i,k_a} using the neural distinguisher. In this work, we use the 7-round distinguisher ND_1^7 and 6-round distinguisher ND_1^6, respectively. We set $N_{ucb} = 500$, $N_{kc} = 5$, $N = 100$, $m = 64$, and $n_k = 32$. The key recovery attack test is implemented 1000 times and divided in to 10 groups. The results are shown in Table 10.

From Table 10, we can see that the total number of successful key recoveries in our attack reaches 618, resulting in a success rate of 61.8%. This is 9.7% higher than the success rate reported in[10]. The success rate can be further improved by increasing the number of neutral bits. In this experiment, the neutral bits were chosen solely to enable a fair comparison with[10]. The computational complexity of the key recovery attack consists of two rounds of key recovery operations. The $(i-1)$-th round of key recovery is performed only when the score of the guessed key in the i-th round of key recovery exceeds the predefined threshold. Consequently, it is difficult to precisely estimate the computational complexity of the $(i-1)$-th round of key recovery. The computational

Group	1	2	3	4	5	6	7	8	9	10
Number of successes	68	63	64	63	64	64	55	59	59	59

Table 10. The key recovery attack against 11-round Speck32/64.

Sequence number of key recoveries	1	2	3	4	5	6	7	8	9	10
Result	Y	Y	N	Y	N	Y	N	N	Y	Y
Number of used iterations	7956	7778	8192	1203	8192	8192	8192	8192	2490	1755
Sequence number of key recoveries	11	12	13	14	15	16	17	18	19	20
Result	Y	N	Y	Y	Y	Y	N	N	Y	Y
Number of used iterations	1722	8192	4165	648	6177	8192	8192	8192	1689	692

Table 11. The key recovery attack against 13-round Speck32/64.

complexity of the i-th round of key recovery is approximately $O\left(N_{ucb} * N_{kc} * n_k * O(ND\,(\,))\right)$. Thus, the primary difference between the proposed method and Gohr's method lies in the inference complexity of the neural distinguisher. The execution time of a single attack was tested, showing that the proposed scheme requires 1500 s, whereas the method in[10] takes 500 s. Although the proposed key recovery attack requires more time, it remains acceptable and offers a significant advantage in terms of success rate.

Furthermore, we analyze the impact of ND_1's accuracy on the success rate of the key recovery attack. As shown in Algorithm 1, scoring the decrypted ciphertext $E_{k_a}^{-1}(X_i)$ is the core operation in key recovery. The score for each possible key is defined in formula (8), where each key's score is obtained using ND_1. For example, in the 11 round-reduced Speck 32/64, if ND_1 outputs a predicted score 0.512 for all 64 ciphertext pairs with a fixed input difference, its final score is $w_{i,k_a} = \log_2 \frac{0.512}{1-0512} = 0.06926$ and $w_k = \sum_{i=1}^{64} w_{i,k_a} = 4.4328$. Suppose the predicted score is improved to 0.513, then we obtain $w_{k'} = 4.8024$. For the 7 round-reduced Speck 32/64, if w_{i,k_a} is improved from 0.788 to 0.7904, the total score w_k increases from 121.2244 to 122.5564. The larger the w_k, the higher the success rate of the key recovery attack. Therefore, improving the accuracy of the neural distinguisher can significantly enhance the success rate of the key recovery attack.

Additionally, we used the method described in[11] to train an 8-round distinguisher for Speck32/64. During the training process, we first utilized 10^7 ciphertext pairs (x_5, x_5^*) of 5-round Speck32/64 with an input difference of $(0x8000{,}0x840a)$ to retrain the 7-round distinguisher ND_1^7 for 10 epochs with a batch size of 500 and a learning rate of 10^{-4}. Here, $(0x8000{,}0x840a)$ is the most likely difference to appear three rounds after the input difference $(0x4000{,}0x0000)$. Subsequently, we train the distinguisher ND_1^8 using 2×10^7 ciphertext pairs (x_8, x_8^*) of 8-round Speck32/64 with an input difference of $(0x4000{,}0x0000)$ for 10 epochs with a batch size of 10,000 and a learning rate of 10^{-4}. Finally, we retrain the distinguisher ND_1^8 using another 2×10^7 ciphertext pairs (x_8, x_8^*), this time with a reduced learning rate of 10^{-5}. The accuracy of the new distinguisher ND_1^8 is test, yielding a result of 51.38%. Base on the distinguishers ND_1^8 and ND_1^7, we implement the same key recovery attack for 13-round Speck32/64 with neutral bits $\{20{,}13{,}22, \{12{,}19\}, \{14{,}21\}, \{6{,}29\}, 30, \{0{,}8{,}31\}, \{5{,}28\}, \{15{,}24\}, \{6{,}11{,}12{,}18\}, \{4{,}27{,}29\}, \}$ and number of ciphertext structures of $N = 2^{11}$. Each ciphertext structure consists of 2^{12} ciphertext pairs. The key recovery attack is only tested 20 times and the result is shown in Table 11.

From Table 11, we can see that the success rate of the key recovery attack against the 13-round reduced Speck32/64 using the proposed neural distinguisher is 65%. In contrast, this attack cannot be implemented by the methods described in[10,20,33]. Therefore, the proposed neural distinguisher demonstrates significant advantages in key recovery attacks.

Conclusion

To improve the accuracy of the neural distinguisher based on deep learning, we explore neural distinguishers using different convolutional block models. A neural distinguisher based on dense residual connections and multiple convolutional layers is proposed. The convolutional block module with parallel convolutional layers and an initial convolutional module using multi-scale convolutional kernel sizes are designed to validate the efficiency of the proposed neural distinguisher. The experiments show that adding incremental convolutional layers and adjusting the convolutional kernel sizes within and between the residual blocks (with a step of 2) can significantly improve accuracy. However, the initial convolutional module using multi-scale convolutional kernel sizes does not improve accuracy. Additionally, we constructed two new datasets, which suggest that the combination of ciphertext pairs, decryption keys, and the differences between ciphertext pairs from different decryption rounds can significantly improve the accuracy of the neural distinguisher. The key recovery attack results show that the proposed neural distinguisher has a significant advantage.

Although the proposed neural distinguisher using 256 ciphertext pairs achieves higher accuracy, it cannot yet be used to recover the key directly. The reason is that, to ensure the generated correct ciphertext structure follows the same distribution for the two rounds of differential paths $(0x211{,}0xa04) \to (0x40{,}0)$, $\log_2 256 = 8$ neutral bits are required to extend a ciphertext pair structure. However, there are only 12 neutral and probabilistic neutral bits available, which can generate only 16 ciphertext pair structures, significantly reducing the success rate of the key recovery attack. This issue will be the focus of future work. Finding more neutral and probabilistic neutral bits may help mitigate this problem.

Data availability

Data will be made available on request from the corresponding author.

Received: 5 February 2025; Accepted: 10 April 2025

Published online: 21 April 2025

References

1. Begum, M. B. et al. Dynamic network security leveraging efficient covinet with Granger causality-inspired graph neural networks for data compression in cloud IoT devices. *Knowl. Based Syst.* **309**, 112859 (2025).
2. Begum, M. B. et al. An enhanced heterogeneous local directed acyclic graph blockchain with recalling enhanced recurrent neural networks for routing in secure MANET-IOT environments in 6G. *Int. J. Commun Syst*. **38**(4), e6110 (2025).
3. Begum, M. B. & Kaliyaperumal, K. Integration of BWT scrambling and data compression in an innovative system enhances protection and versatile management of sensor feeds (SEC). *Heliyon* **10**(20) (2024).
4. Begum, M. B. et al. Cereal Crop Variety Classification using Deep Learning.
5. Begum, M. B. et al. Enhancing agricultural productivity with data-driven crop recommendations. *2023 Second International Conference on Augmented Intelligence and Sustainable Systems (ICAISS)* 1055–1063 (IEEE, 2023).
6. Wang, Y. et al. FN-HNN coupled with tunable multistable memristors and encryption by arnold mapping and diagonal diffusion algorithm. *IEEE Trans. Circuits Syst. I* (2024).
7. Dou, G. et al. Operant conditioning neuromorphic circuit with addictiveness and time memory for automatic learning. *IEEE Trans. Biomed. Circuits Syst.* (2024).
8. Sun, J. et al. Memristor-based parallel computing circuit optimization for LSTM network fault diagnosis. *IEEE Trans. Circuits Syst. I* (2024).
9. Xia, H. et al. Capacitor parameter Estimation based on wavelet transform and Convolution neural network. *IEEE Trans. Power Electron.* (2024).
10. Gohr, A. Improving attacks on round-reduced speck32/64 using deep learning. Advances in Cryptology–CRYPTO. *39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part II 39* 150–179 (Springer International Publishing, 2019).
11. Beaulieu, R. et al. The SIMON and SPECK lightweight block ciphers. *Proceedings of the 52nd Annual Design Automation Conference* 1–6 (2015).
12. Benamira, A. et al. A deeper look at machine learning-based cryptanalysis. Advances in Cryptology–EUROCRYPT. *40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17–21, 2021, Proceedings, Part I 40* 805–835 (Springer International Publishing, 2021).
13. Bellini, E., Rossi, M. & Conference Computing Performance comparison between deep learning-based and conventional cryptographic distinguishers. Intelligent Computing: Proceedings of the Volume 3 681–701 (Springer International Publishing, 2021).
14. Tian, W. & Hu, B. Deep learning assisted differential cryptanalysis for the lightweight cipher Simon. *KSII Trans. Internet Inform. Syst. (TIIS)*. **15**(2), 600–616 (2021).
15. Hou, Z., Ren, J. & Chen, S. *Improve Neural Distinguisher for Cryptanalysis* (Cryptology ePrint Archive, 2021).
16. Hou, Z., Ren, J. & Chen, S. *Cryptanalysis of Round-Reduced SIMON32 Based on Deep Learning* (Cryptology ePrint Archive, 2021).
17. Su, H. C., Zhu, X. Y. & Ming, D. Polytopic attack on round-reduced simon32/64 using deep learning. *Information Security and Cryptology: 16th International Conference, Inscrypt 2020, Guangzhou, China, December 11–14, 2020, Revised Selected Papers* 3–20 (Springer International Publishing, 2021).
18. Wang, G., Wang, G. & He, Y. Improved machine learning assisted (related-key) differential distinguishers for lightweight ciphers. *2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)* 164–171 (IEEE, 2021).
19. Zhang, R. et al. Differential Cryptanalysis of TweGIFT-128 Based on Neural Network. *2021 IEEE Sixth International Conference on Data Science in Cyberspace (DSC)* 529–534 (IEEE, 2021).
20. Lyu, L., Tu, Y. & Zhang, Y. Improving the deep-learning-based differential distinguisher and applications to Simeck. *2022 IEEE 25th International Conference on Computer Supported Cooperative Work in Design (CSCWD)* 465–470 (IEEE, 2022).
21. Rajan, R. et al. Deep learning-based differential distinguisher for lightweight cipher GIFT-COFB. *Machine Intelligence and Smart Systems: Proceedings of MISS 2021* 397–406 (Springer Nature Singapore, 2022).
22. Mishra, G. et al. Deep learning-based differential distinguisher for lightweight ciphers GIFT-64 and PRIDE. *Machine Intelligence and Smart Systems: Proceedings of MISS 2021* 245–257 (Springer Nature Singapore, 2022).
23. Baksi, A. & Baksi, A. Machine learning-assisted differential distinguishers for lightweight ciphers. *Classical Phys. Secur. Symmetric Key Cryptographic Algorithms* 141–162 (2022).
24. Bao, Z. et al. Enhancing differential-neural cryptanalysis. *International Conference on the Theory and Application of Cryptology and Information Security* 318–347 (Springer Nature Switzerland, 2022).
25. Zhang, L. & Wang, Z. Improving differential-neural distinguisher model for DES, Chaskey, and PRESENT. arXiv preprint arXiv:2204.06341 (2022).
26. Chen, Y. & Yu, H. A new neural distinguisher model considering derived features from multiple ciphertext pairs. *IACR Cryptol.* 310 (2021).
27. Chen, Y., Shen, Y. & Yu, H. Neural-Aided statistical attack for cryptanalysis. *Comput. J.* **66**(10), 2480–2498 (2023).
28. Zhang, L. & Wang, Z. *Improving Differential-Neural Cryptanalysis* (Cryptology ePrint Archive, 2022).
29. Chen, Y. et al. A new neural distinguisher considering features derived from multiple ciphertext pairs. *Comput. J.* **66**(6), 1419–1433 (2023).
30. Gohr, A., Leander, G. & Neumann, P. *An Assessment of differential-neural Distinguishers* (Cryptology ePrint Archive, 2022).
31. Hou, Z., Ren, J. & Chen, S. Practical attacks of round-reduced Simon based on deep learning. *Comput. J.* **66**(10), 2517–2534 (2023).
32. Bao, Z. et al. More insight on deep learning-aided cryptanalysis. *International Conference on the Theory and Application of Cryptology and Information Security* 436–467 (Springer Nature Singapore, 2023).
33. Liu, J. S. et al. Improved neural distinguishers with multi-round and multi-splicing construction. *J. Inform. Secur. Appl.* **74**, 103461 (2023).
34. Lu, J. et al. Improved (related-key) differential-based neural distinguishers for SIMON and SIMECK block ciphers. *Comput. J.* **67**(2), 537–547 (2024).
35. Seok, B., Chang, D. & Lee, C. *A Novel Approach To Construct a Good Dataset for differential-neural Cryptanalysis*(IEEE Transactions on Dependable and Secure Computing, 2024).
36. Wang, G., Wang, G. & Sun, S. Investigating and enhancing the neural distinguisher for differential cryptanalysis. *IEICE. Trans. Inf. Syst.* **107**(8), 1016–1028 (2024).
37. Huang, G. et al. Densely connected convolutional networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* 4700–4708 (2017).
38. Chollet, F. et al. https://keras.io (2015).

Acknowledgements

This work was supported by the following projects and foundations: 2024 Shaanxi Province key research and development plan project (No. 2024GX-ZDCYL-01-13), the Special Funds for Basic scientific Research of the

Central universities (No. D5000210638).

Author contributions
Yufei Hou: Writing – review & editing, Validation, Methodology, Investigation, Data curation. Jie Liu: Writing - review & editing, Writing - original Draft, Supervision, Conceptualization, Methodology, Funding acquisition. Shouxu Han: Investigation, Software. Zhongjun Ma: Resources, Validation. Xi Ye: Validation, Software. Xuan Nie: Visualization, Writing - review & editing, Funding acquisition.

Declarations

Competing interests
The authors declare no competing interests.

Additional information
Correspondence and requests for materials should be addressed to J.L.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.